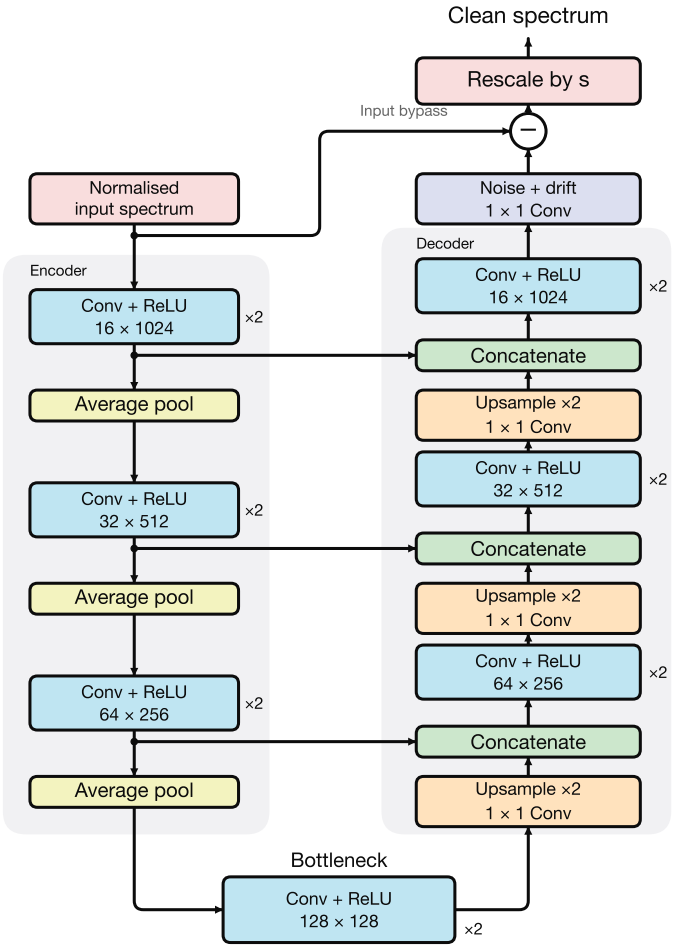
THE UNIVERSITY OF MELBOURNE

The Art of Scientific Computation
COMP90072

Denoising EDMR Spectra Using Machine Learning

Draft report

Reave Paleg
Student ID: 1473206



Each blue block: two Conv1D + ReLU pairs (k = 7)
Dimensions: channels × samples · pooling halves the length

Faculty of Science
The University of Melbourne

7 October 2026

Abstract

Electrically detected magnetic resonance (EDMR) spectra contain random noise and a slowly varying background. Removing these is useful only if the positions, widths and amplitudes of the resonance features are preserved. This project creates a denoising model using synthetic spectra with known clean targets. Each spectrum contains one to five derivative-Gaussian peaks, a random mixture of coloured noise, and a possible smooth baseline drift. A residual one-dimensional U-Net predicts the unwanted noise and drift, which are subtracted from the input. Amplitude normalisation is shared with the clean target and reversed after inference.

The experiment uses 50,000 training spectra, 2,000 validation spectra and 2,000 test spectra. The trained model achieves a validation mean squared error of 0.002015 and a test mean squared error of 0.002039, both in original amplitude units. The corresponding noisy test inputs have a mean squared error of 0.081426. The model is compared with five traditional methods whose settings are chosen using validation data. The strongest traditional method, polynomial baseline correction followed by Savitzky-Golay smoothing, has a test mean squared error of 0.015177. Sample reconstructions show substantial noise and baseline suppression, although some local peak-shape errors and a false peak remain. These results are for synthetic spectra; measured EDMR spectra still need to be evaluated.

Contents

Abstract **i**

1 Denoising EDMR spectra **1**

1.1 Introduction and project aim 1

1.2 Creating the synthetic dataset 1

1.3 The neural network 3

1.4 Results 6

1.5 Discussion and remaining validation 8

1.6 Conclusion 9

Bibliography **10**

Chapter 1

Denoising EDMR spectra

1.1 Introduction and project aim

Electrically detected magnetic resonance (EDMR) measures changes in the current through a device when a magnetic resonance condition is reached. The resulting spectrum provides information about spin-dependent processes in the material. In practice, the measured response also depends on the sample and measurement circuit [3]. The aim of this project is to investigate whether a neural network can recover an ideal spectrum from a noisy measurement without removing resonance features. The general idea is to create a synthetic dataset of clean spectra, add controlled noise and drift, train a model using this dataset, and compare its output with the known clean signal. The model is also compared with traditional denoising techniques. Synthetic data are useful at this stage because the correct answer is known for every spectrum, including spectra with overlapping peaks.

1.2 Creating the synthetic dataset

1.2.1 Clean spectra

First we start with an index grid representing a series of magnetic field strengths $x = 0, 1, \ldots, 1023$. For an EDMR peak, individual components have the shape

$$f(x) = a(x-b)\exp\left[-c(x-b)^2\right], \qquad c > 0. \tag{1.1}$$

This is proportional to the derivative of a Gaussian. Each clean spectrum is the sum of K resonance features,

$$y(x) = \sum_{k=1}^{K} f_k(x), \qquad K \in \{1, 2, 3, 4, 5\}. \tag{1.2}$$

The number of components is sampled uniformly. For each component, the width parameter w is sampled uniformly between 0 and 40 sample intervals, and its individual lobe height h is sampled uniformly between 0 and 1.5. For $w > 0$, the parameters in Equation 1.1 are then set to

$$a = \frac{h\exp(1/2)}{w}, \qquad c = \frac{1}{2w^2}. \tag{1.3}$$

An exactly zero-width component contributes zero on the sampled grid. The extrema occur at $x = b \pm w$, where the component has amplitudes $\pm h$. This makes the chosen width and height directly interpretable. The centre is drawn from $b \sim U(-40, 1063)$, allowing peaks to extend beyond

the scan boundaries. Overlap is allowed, so the extrema of the summed spectrum need not have the height of any individual component. Amplitudes are synthetic units, and x has not been calibrated to magnetic field.

1.2.2 Coloured-noise mixtures

Each spectrum contains a random mixture of zero to five distinct noise colours. I wanted to include multiple types and mixtures of noise in the dataset to help the algorithm generalise better. The colours and their power law exponents are shown in Table 1.1. White noise is sampled from a Gaussian distribution. For the other colours, the real Fourier transform of a white-noise sequence is multiplied by a frequency-dependent amplitude weight,

$$A(f) \propto f^{-\beta/2}, \qquad P(f) \propto f^{-\beta}, \qquad f > 0. \tag{1.4}$$

This Fourier-weighting approach is described in [1]. The zero-frequency coefficient is set to zero. After the inverse transform, the coloured component has its mean subtracted and is multiplied by a factor that gives it the sampled standard deviation.

Colour	β	Relative emphasis
Brown	2	Low frequencies
Pink	1	Low frequencies
White	0	Flat power spectrum
Blue	-1	High frequencies
Violet	-2	High frequencies

Table 1.1: Noise colours implemented in the notebook. The mixture need not follow one power law.

The selected components receive independent strength settings from $U(0, 1.0)$ and are summed. The resulting mixture is centred and rescaled to an overall standard deviation $\sigma \sim U(0, 0.4)$. The relative mixture strengths and total noise level therefore vary separately. Zero selected components or zero total strength produce no random noise. Each sample has fixed noise once the dataset is created; the noise is not regenerated at every training epoch.

1.2.3 Baseline drift

A separate smooth baseline drift is added to 80% of the spectra. The remaining 20% have no added baseline drift. Let u run uniformly from 0 to 1 across the scan, and let $t = 2u - 1$. The drift is

$$d(x) = d_0 + d_1 t + d_2 \left(t^2 - \overline{t^2}\right) + A \sin(2\pi\nu u + \phi). \tag{1.5}$$

The offset and slope are each drawn uniformly between -0.3 and 0.3. Curvature lies between -0.2 and 0.2, the sinusoidal amplitude between 0 and 0.15, and the number of cycles between 0 and 1.5. Phase is uniform between 0 and 2π.

The corrupted input is

$$z(x) = y(x) + n(x) + d(x), \tag{1.6}$$

where n is the coloured-noise mixture. The target remains $y(x)$, so both noise and baseline drift must be removed. Figure 1.1 illustrates the generated contributions.

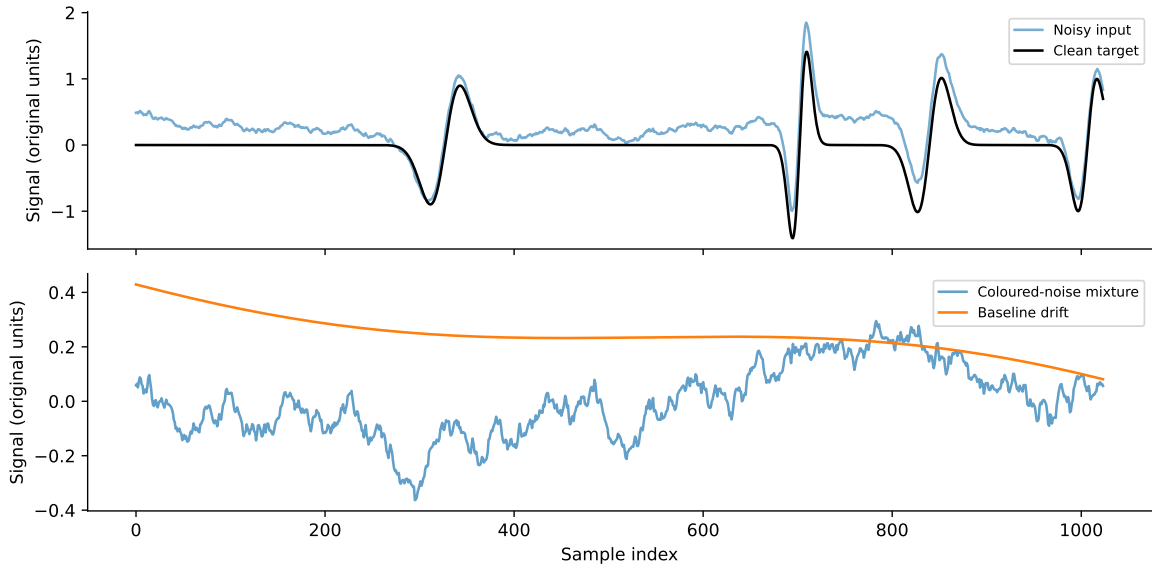


Figure 1.1: A synthetic example. The upper graph shows the clean target and corrupted input. The bottom graph shows the coloured-noise mixture and baseline drift.

1.2.4 Normalisation and data splits

The input amplitude is normalised separately for each spectrum. The scale is the larger of the 95th percentile of the absolute corrupted input and one-tenth of its maximum absolute amplitude,

$$s_i = \max\left[Q_{0.95}(|z_i|)\,,\, \frac{\max_j |z_{ij}|}{10}\right], \qquad \widetilde{z}_i = \frac{z_i}{s_i}, \qquad \widetilde{y}_i = \frac{y_i}{s_i}. \tag{1.7}$$

The maximum-amplitude term prevents the normalised input from becoming extremely large and overflowing the training loss.

The code generates 50,000 training spectra with seed 10, 2,000 validation spectra with seed 20, and 2,000 test spectra with seed 30. Performance on the validation set is used to select the best model weights. The test set is used for the final evaluation and reconstruction plots.

1.3 The neural network

1.3.1 Residual one-dimensional U-Net

Initially I tested many types of neural network implementations, including auto-encoders and convolutional neural networks. The network that worked best ended up being the design introduced in U-Net [4] which combines ideas used in both auto-encoders and CNNs. The encoder reduces the spatial resolution to learn broader patterns and filter out noise, and the decoder restores the original resolution. At each decoder level, features from the matching encoder level are concatenated. These skip connections allow for preservation of finer information.

The network predicts the unwanted contribution r_θ, and returns

$$\widehat{\widetilde{y}} = \widetilde{z} - r_\theta(\widetilde{z}). \tag{1.8}$$

Predicting the unwanted contribution follows the residual-denoising idea used in DnCNN [8]. Here the residual includes both the random noise and the added drift. The model's targets are the clean outputs. The final output layer has no activation, allowing the residual to have either sign. Its weights and bias are initially zero, so the untrained model starts as an identity mapping.

Each block applies two sets of learned filters to detect patterns in the spectrum. A convolution is like a matrix multiplication, input values are multiplied by learned weights and added together. The same weights are reused as the filter moves along the spectrum. Each filter examines seven neighbouring points at a time (kernel size 7). Three zeros are added at each end of the spectrum so the filter can go over the endpoints of the spectrum. After each filtering step, negative values are set to zero, this operation is called ReLU, and allows the network to learn non-linear patterns.

The encoder gradually reduces the size of the representation by averaging neighbouring pairs of values (average pooling). At the levels, it produces 16, 32, 64 and 128 feature channels, intended to learn different patterns.

The decoder increases the representation length by estimating values between existing points (linear interpolation). A convolution then performs a matrix multiplication across the channels at each position, mixing the information they contain. These traces are joined with information saved from the matching encoder level (skip connections), then passed through another filtering block. The network has 352,177 adjustable parameters, which are learned during training. Figure 1.2 shows the architecture structure.

The final output is obtained by multiplying by the saved input scale,

$$\widehat{y}_i = s_i \widehat{\widetilde{y}}_i. \tag{1.9}$$

1.3.2 Loss, optimiser and early stopping

The training loss is the mean squared error (MSE) between the normalised prediction and target. For M spectra with $N = 1024$ points,

$$\mathcal{L} = \frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} \left(\widehat{\widetilde{y}}_{ij} - \widetilde{y}_{ij} \right)^2. \tag{1.10}$$

A smaller value indicates a closer match to the known target, and zero would indicate a perfect pointwise reconstruction. Normalised MSE weights the original squared errors by $1/s_i^2$, so it is not the same quantity as original-unit MSE. The model is trained in PyTorch with Adam [2] at a learning rate of 10^{-3} and a batch size of 128. The maximum is 40 epochs. Early stopping is applied when validation MSE has not improved by more than 10^{-5} relative to the patience reference for six epochs. The best validation weights are saved independently of this threshold and restored at the end. Training uses gradient updates; validation uses evaluation mode without gradient tracking. The model was trained on Apple's MPS.

1.3.3 Traditional methods

The model is compared with a moving average [6], Gaussian smoothing [5], Savitzky-Golay smoothing [7], polynomial baseline correction, and polynomial baseline correction followed by Savitzky-Golay smoothing. The smoothing methods average nearby samples or fit a local polynomial. Baseline correction fits a polynomial across the noisy scan and subtracts it before any smoothing. It uses only the noisy input, not the clean target. These methods were inspiration for the neural design I used, but the machine learning implementation allows these smoothing ideas to be fine tuned to EDMR spectra in particular.

Each method is tuned using normalised validation MSE, same as for the neural network. The settings in Table 1.2 are then fixed for test evaluation.

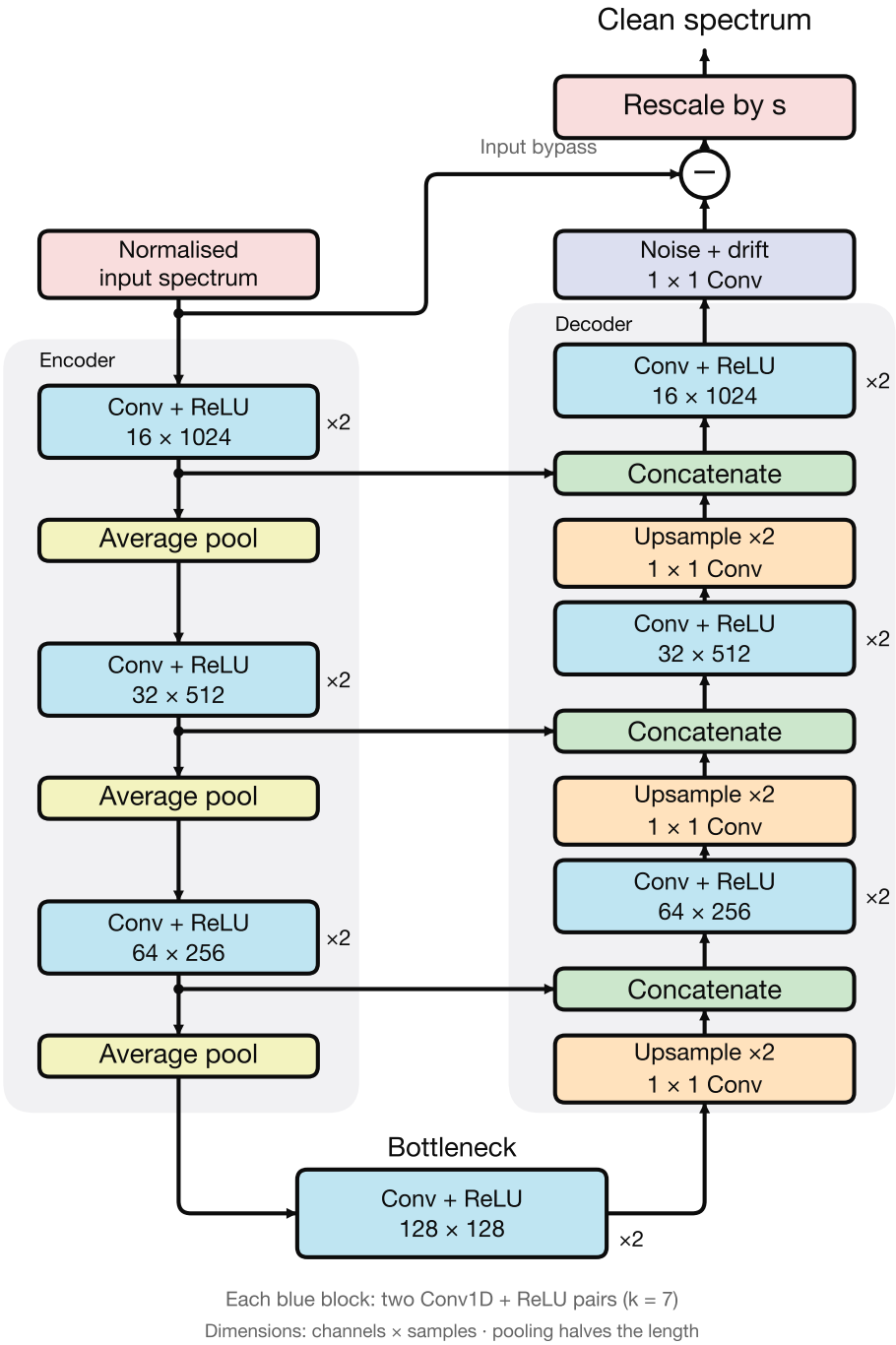


Figure 1.2: The implemented residual 1D U-Net. Dimensions are output channels × sequence length. Each blue block contains two Conv1D–ReLU pairs in series, with separate learned weights. Horizontal skip connections concatenate matching encoder features. The final kernel-size-1 convolution predicts noise and drift; subtraction and inverse scaling give the clean prediction in original amplitude units. AI assistance was used to construct this diagram.

Method	Selected settings
Moving average	Window 11 samples
Gaussian smoothing	Standard deviation 4 samples
Savitzky-Golay	Window 21 samples; polynomial order 2
Polynomial baseline	Degree 3
Baseline + Savitzky-Golay	Baseline degree 3; window 21 samples; order 2

Table 1.2: Traditional-method settings selected using validation data.

1.4 Results

1.4.1 Training history

The training run completed in 26 epochs and restored the epoch-23 checkpoint selected using the internal normalised validation loss. Its validation MSE in original amplitude units is 0.002015. The saved training history is shown in Figure 1.3. Reported MSE values are in original amplitude units squared.

Quantity	Training MSE	Validation MSE
Epoch 1	0.018661	0.003940
Restored epoch 23	0.001841	0.002015
Final epoch 26	0.001769	0.002171

Table 1.3: MSE in original amplitude units squared from the training run.

The error falls rapidly at the start of training and then improves slowly. Between epochs 23 and 26, training loss decreases while validation loss fluctuates above its minimum. This is consistent with diminishing generalisation and late overfitting. Early stopping limits further training when validation performance stops improving.

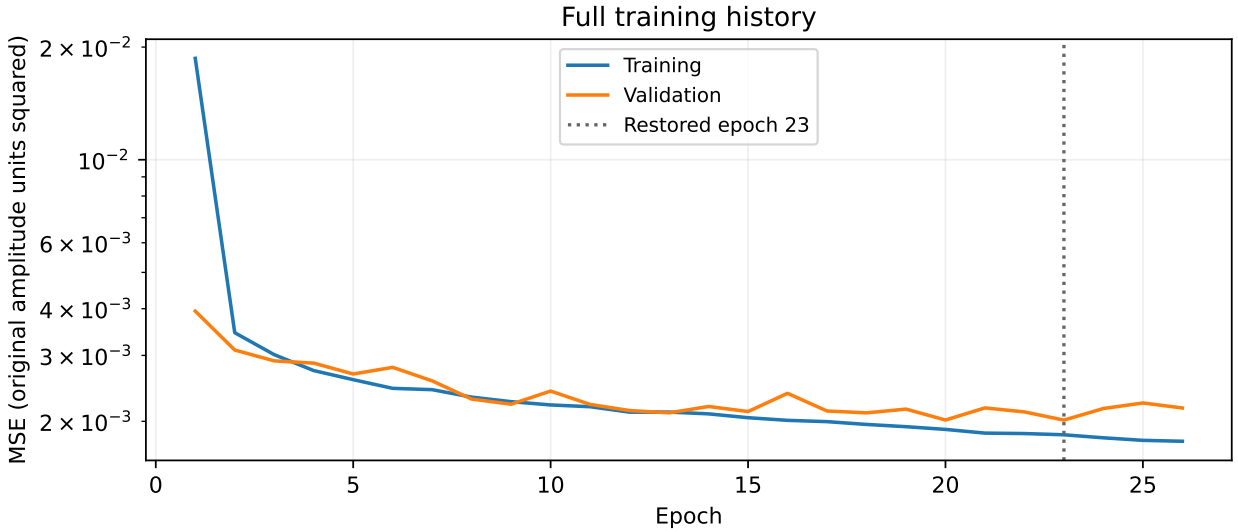


Figure 1.3: Training and validation MSE in original amplitude units squared versus epoch. The MSE is on a logarithmic axis. The dotted line marks the selected network weights from early stopping at epoch 23.

1.4.2 Test evaluation and comparison with the corrupted input

The restored model is evaluated on the 2,000 unseen test spectra. Its test MSE is 0.002039 in original amplitude units squared, compared with 0.081426 for the unprocessed input. The input and prediction are compared with the same clean target after reversing the input normalisation. The test set does not select the checkpoint or filter settings, so this is an unseen test set evaluation.

1.4.3 Example reconstructions

Figure 1.4 compares the network with polynomial baseline correction followed by Savitzky-Golay smoothing on the first three test spectra. These fixed examples were not selected for good results. Predictions have been de-normalised, so the curves can be compared in the original synthetic amplitude units. The predictions suppress substantial noise and drift. In the second and third examples, the predicted and clean curves are close over much of the resonance feature. The first example still shows amplitude differences, baseline wiggles and a false peak where the clean target is flat. These examples support qualitative reconstruction of the generated features, but also show why a low average MSE does not guarantee that every reconstructed peak is correct.

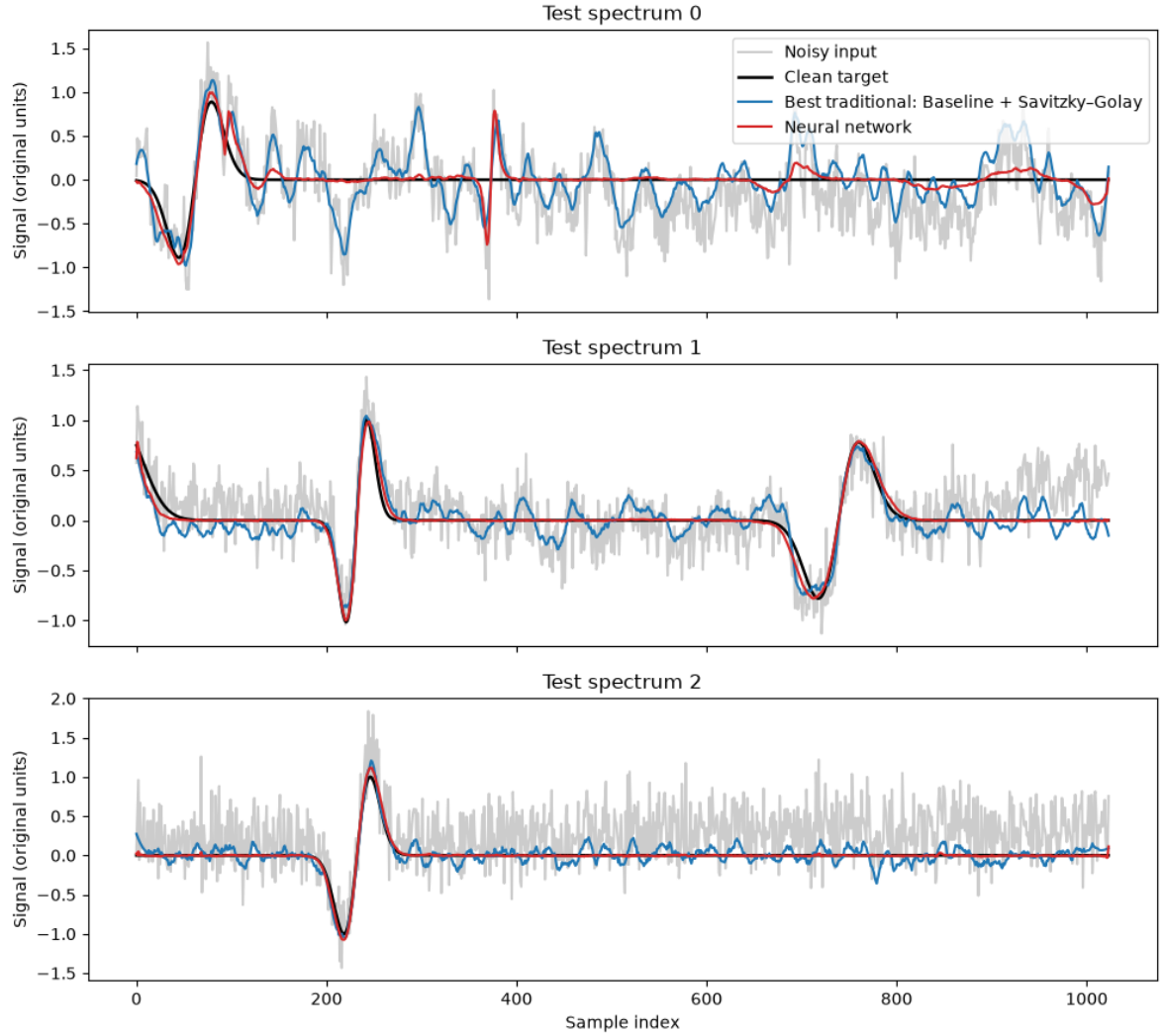


Figure 1.4: Comparison on test spectra. Grey is the corrupted input, black is the clean target, blue is polynomial baseline correction followed by Savitzky-Golay smoothing, and red is the restored network output. All curves are in original synthetic amplitude units.

1.4.4 Comparison with traditional techniques

Table 1.4 compares every method on the same test spectra. Polynomial baseline correction followed by Savitzky-Golay smoothing is the strongest traditional method by MSE. Its original-unit MSE is 0.015177, compared with 0.002039 for the neural network. Figure 1.5 shows the original-unit errors.

Method	MSE
Noisy input	0.081426
Moving average	0.052340
Gaussian smoothing	0.051858
Savitzky-Golay	0.052813
Polynomial baseline	0.043791
Baseline + Savitzky-Golay	0.015177
Neural network	0.002039

Table 1.4: MSE in original amplitude units squared on the same 2,000 held-out test spectra. Values are rounded from the current notebook's saved comparison table.

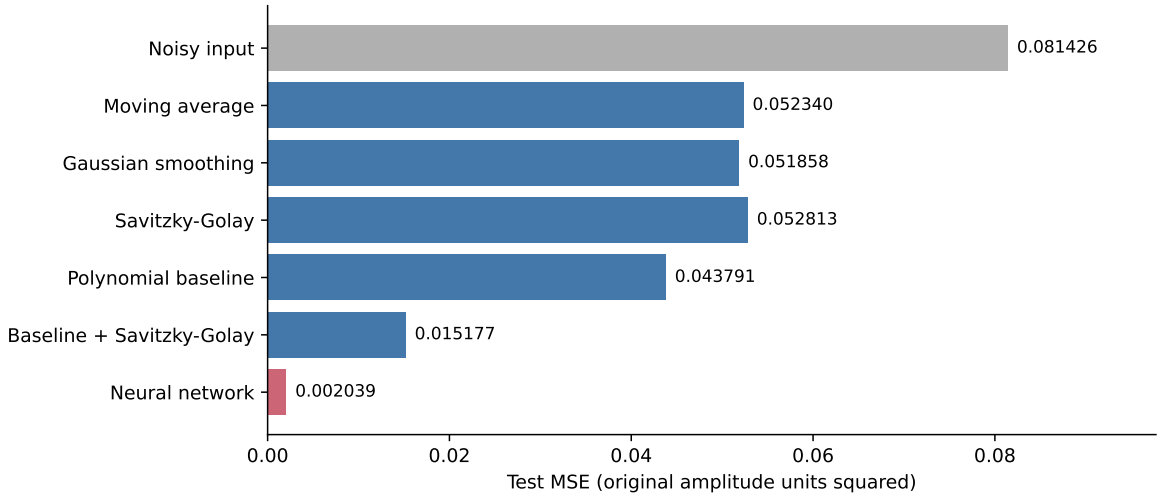


Figure 1.5: Original-unit MSE on the same held-out test set. Lower values mean closer agreement with the clean target.

1.5 Discussion and remaining validation

1.5.1 Peak preservation and limitations of MSE

Pointwise MSE is a useful overall measure, but it does not directly report errors in peak position, lobe height or linewidth. Error over many near-baseline samples can dominate the average metric, while an isolated false peak can still be important. The next evaluation should quantify these feature errors. A peak-preserving loss or explicit line fitting may be worth investigating. An alternative model trained to predict the input feature parameters, such as feature position, lobe height and linewidth, is also worth investigating.

1.5.2 Comparison with traditional methods

The test comparison shows lower aggregate errors for the network than for the five traditional methods tested here. Baseline correction improves the traditional smoothing result substantially, because smoothing alone leaves slow drift. A baseline polynomial can also remove part of a broad or edge-truncated resonance. The comparison covers a finite search of filter settings on one synthetic distribution. It does not show that the network is better than every traditional method or that the same advantage will hold for measured spectra.

1.5.3 Application to measured EDMR spectra

For measured spectra, the input should be represented on a known, consistently ordered magnetic-field grid. The network currently operates on sample indices, so its notion of width is in points. A physical width w_B corresponds to approximately $w_B/\Delta B$ sample intervals when the field spacing is ΔB. Changing the number of points without accounting for the field spacing can therefore change the features seen by the model.

Amplitude normalisation is already included, but it does not correct all differences between synthetic and measured data. These assumptions limit the present scope. A useful real-data validation would compare individual noisy scans with an independently acquired higher-SNR reference, while checking that denoising does not alter fitted feature parameters. Reference acquisition and scan alignment also need to be considered. No measured EDMR spectrum has been processed or validated in the current notebook.

1.6 Conclusion

The current project implements a synthetic denoising pipeline with variable peak counts and strengths, mixed coloured noise, smooth baseline drift, and a residual 1D U-Net. The trained model's validation MSE is 0.002015 and its test MSE is 0.002039, both in original amplitude units squared. The example predictions show substantial removal of the generated corruption, with local reconstruction errors still visible in the first test spectrum.

The experiment is a proof of concept. On the held-out test set, the network has an MSE of 0.002039, compared with 0.015177 for polynomial baseline correction followed by Savitzky-Golay smoothing, the strongest traditional method tested. Future experiments are quantitative measures of peak preservation, a model that predicts peak parameters, repeated training seeds, and validation on real EDMR scans.

Bibliography

[1] Bob. Generate colors of noise in Python. Stack Overflow answer, 2021. Posted 16 April 2021; accessed 8 October 2026. https://stackoverflow.com/questions/67085963/generate-colors-of-noise-in-python.

[2] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, 2015. https://arxiv.org/abs/1412.6980.

[3] D. M. G. Leite, A. Batagin-Neto, O. Nunes-Neto, J. A. Gómez, and C. F. O. Graeff. Electrically detected magnetic resonance modeling and fitting: An equivalent circuit approach. arXiv:1307.1022, 2013. https://arxiv.org/abs/1307.1022.

[4] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. arXiv:1505.04597, 2015. https://arxiv.org/abs/1505.04597.

[5] The SciPy community. *SciPy: `scipy.ndimage.gaussian_filter1d`*. SciPy. API documentation; accessed 8 October 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.gaussian_filter1d.html.

[6] The SciPy community. *SciPy: `scipy.ndimage.uniform_filter1d`*. SciPy. API documentation; accessed 8 October 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.uniform_filter1d.html.

[7] The SciPy community. *SciPy: `scipy.signal.savgol_filter`*. SciPy. API documentation; accessed 8 October 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.savgol_filter.html.

[8] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang. Beyond a gaussian denoiser: Residual learning of deep CNN for image denoising. IEEE Transactions on Image Processing, 2017. doi:10.1109/TIP.2017.2662206; https://arxiv.org/abs/1608.03981.